OYE NINO

SQL EXPLAINED

LIKE A FRIEND

The Beginner's SQL Cheat Sheet with Practical Examples

PostgreSQL Edition — Version 1.0

No complicated language. No prior database knowledge. Just a college database story, real SQL examples, and explanations you can actually understand and try.

oyenino.com

Instagram: @oye.nino — YouTube: @oye_nino

> **The story:** Rahul (student ID 101) and Priya (student ID 102) study at the same college. The college keeps student details in one table and relationship details in another. We'll use these two tables to understand how databases work.

How to use this guide: Read each "Imagine this" explanation, look at the SQL command, then read "What just happened?" before moving on. Try the exercises near the end. **SQL dialect:** PostgreSQL.

Part 1 — Before SQL, understand the database

1. What is a database?

> **Imagine this:** Your college has hundreds of students. Their names, email addresses, courses and other records cannot just live on random sticky notes. You need an organized place to keep them.

A **database** is that organized place. A **table** is like a register inside the database.

- **Column** = a type of detail, such as `first_name`.
- **Row** = one complete record, such as Rahul's details.
- **Cell** = a single value, such as `Rahul`.

Our Students table

ID	FIRST_NAME	EMAIL
101	Rahul	rahul@college.test
102	Priya	priya@college.test
103	Aman	aman@college.test

2. Primary Key (PK): one unique identity

> **Imagine this:** Two students can both be named Rahul. How do you tell them apart? Give each student a unique ID.

`students.id` is the **Primary Key**. It uniquely identifies each student row, and cannot be `NULL`.

3. Foreign Key (FK): connect two tables

Relationships table

ID	STUDENT_A_ID	STUDENT_B_ID	RELATIONSHIP_STATUS	IS_DELETED
1	101	102	dating	false

`student_a_id` and `student_b_id` are **Foreign Keys** — they point to primary-key values in the Students table.

> **Important:** A primary key does not magically become a foreign key. A separate column in another table stores a reference to it.

4. What is SQL?

SQL = Structured Query Language. It's how we ask a relational database questions and tell it what to change.

- "Show me all students" → `SELECT`
- "Add a new student" → `INSERT`
- "Change a relationship status" → `UPDATE`
- "Remove a row" → `DELETE`

Part 2 — Set up a safe practice database

> **Only run this in a new, disposable PostgreSQL database.**

```
CREATE TABLE students (
  id INTEGER PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE relationships (
  id INTEGER PRIMARY KEY,
  student_a_id INTEGER NOT NULL REFERENCES students(id),
  student_b_id INTEGER NOT NULL REFERENCES students(id),
  relationship_status VARCHAR(20) NOT NULL,
  is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
  ended_at TIMESTAMPTZ,
  CHECK (student_a_id <> student_b_id)
);

INSERT INTO students (id, first_name, email) VALUES
  (101, 'Rahul', 'rahul@college.test'),
  (102, 'Priya', 'priya@college.test'),
  (103, 'Aman', 'aman@college.test');

INSERT INTO relationships
  (id, student_a_id, student_b_id, relationship_status, is_deleted)
VALUES
  (1, 101, 102, 'dating', FALSE);
```

Part 3 — Read data with SELECT (DQL)

5. SELECT: "Show me the data"

Imagine this: You walk into the college office and say, "Show me the student register."

```
SELECT * FROM students;
```

What just happened? `SELECT` retrieves data. `*` means all columns. `FROM students` tells SQL which table.

Choose only the columns you need

```
SELECT id, first_name FROM students;
```

6. WHERE: "Only the student I asked for"

```
SELECT * FROM students
WHERE id = 101;
```

What just happened? `WHERE` filters rows. Only Rahul's record matches ID 101.

```
SELECT * FROM students
WHERE first_name = 'Priya';
```

Remember: Text uses single quotes. Numbers generally don't need them.

7. AND / OR / NOT: combine conditions

```
SELECT * FROM students
WHERE id ⩾ 101 AND id ⩽ 102;
```

```
SELECT * FROM students
WHERE first_name = 'Rahul' OR first_name = 'Aman';
```

8. ORDER BY: arrange the results

```
SELECT id, first_name FROM students
ORDER BY first_name ASC;
```

What just happened? Results sorted A → Z. Use `DESC` for reverse.

9. LIMIT and OFFSET: show a small page

```
SELECT id, first_name FROM students
ORDER BY id
LIMIT 2 OFFSET 0;
```

What just happened? Returns the first two students. `OFFSET 2` would skip those and show the next page.

10. LIKE and ILIKE: search by text

```
SELECT * FROM students
WHERE first_name ILIKE 'ra%';
```

What just happened? PostgreSQL `ILIKE` is case-insensitive. `%` matches any characters. Rahul matches.

11. IN and BETWEEN: shorthand filters

```
SELECT * FROM students
WHERE id IN (101, 103);
```

```
SELECT * FROM students
WHERE id BETWEEN 101 AND 102;
```

12. IS NULL: missing information

```
SELECT * FROM relationships
WHERE ended_at IS NULL;
```

What just happened? Finds records with no end date. Use `IS NULL`, not `= NULL`.

Part 4 — Read related data with JOIN

13. JOIN: connect IDs to names

> **Imagine this:** The Relationships table says `101` and `102`. You want the actual names.

```
SELECT
  a.first_name AS student,
  b.first_name AS partner,
  r.relationship_status AS status
FROM relationships AS r
JOIN students AS a ON r.student_a_id = a.id
JOIN students AS b ON r.student_b_id = b.id
WHERE r.id = 1 AND r.is_deleted = FALSE;
```

Expected result:

STUDENT	PARTNER	STATUS
Rahul	Priya	dating

INNER JOIN vs LEFT JOIN

- **INNER JOIN** (plain `JOIN`) returns rows only when matching records exist on both sides.
- **LEFT JOIN** keeps every row from the left table, even if the right table has no match.

```
SELECT s.first_name, r.id AS relationship_id
FROM students AS s
LEFT JOIN relationships AS r
  ON r.student_a_id = s.id AND r.is_deleted = FALSE
ORDER BY s.id;
```

Part 5 — Change data with DML

14. INSERT: add a new record

```
INSERT INTO students (id, first_name, email)
VALUES (104, 'Neha', 'neha@college.test');
```

15. UPDATE: change existing information

```sql
UPDATE relationships
SET relationship_status = 'broken_up',
    is_deleted = TRUE,
    ended_at = CURRENT_TIMESTAMP
WHERE id = 1;
```

Very important: `UPDATE` without `WHERE` can modify every row in the table.

16. Soft delete vs hard delete

Soft delete: Mark inactive with a flag like `is_deleted`. **Hard delete:** `DELETE FROM` removes the row.

```sql
-- Normal active view
SELECT * FROM relationships WHERE is_deleted = FALSE;

-- Hard delete (DANGEROUS)
DELETE FROM relationships WHERE id = 1;
```

17. DELETE: removing a row is not the same as DROP

```sql
DELETE FROM students WHERE id = 104;
```

This deletes Neha's row. It does **not** delete the Students table itself.

Part 6 — Count and summarize records

18. COUNT

```sql
SELECT COUNT(*) AS total_students FROM students;
```

19. GROUP BY

```sql
SELECT relationship_status, COUNT(*) AS total
FROM relationships
GROUP BY relationship_status;
```

20. HAVING: filter the groups

```sql
SELECT relationship_status, COUNT(*) AS total
FROM relationships
GROUP BY relationship_status
HAVING COUNT(*) ⩾ 1;
```

`WHERE` filters rows before grouping. `HAVING` filters groups after.

21. DISTINCT

```sql
SELECT DISTINCT relationship_status FROM relationships;
```

Part 7 — The five common SQL categories

CATEGORY	FULL FORM	MEANING	COMMANDS
DQL	Data Query Language	Show me something	SELECT
DML	Data Manipulation Language	Change the records	INSERT, UPDATE, DELETE
DDL	Data Definition Language	Build/change the registers	CREATE, ALTER, DROP, TRUNCATE
DCL	Data Control Language	Who is allowed?	GRANT, REVOKE
TCL	Transaction Control Language	Save or undo changes	COMMIT, ROLLBACK, SAVEPOINT

22. DDL: CREATE, ALTER, DROP, TRUNCATE

```
ALTER TABLE students ADD COLUMN enrolled_year INTEGER;
```

23. DCL: GRANT and REVOKE

```
GRANT SELECT ON students TO college_reader;
REVOKE SELECT ON students FROM college_reader;
```

24. TCL: BEGIN, COMMIT, ROLLBACK

```
BEGIN;
UPDATE relationships
SET relationship_status = 'dating',
    is_deleted = FALSE, ended_at = NULL
WHERE id = 1;
SELECT * FROM relationships WHERE id = 1;
ROLLBACK;
-- Or use COMMIT to save the change.
```

SAVEPOINT

```
BEGIN;
SAVEPOINT before_change;
UPDATE relationships SET is_deleted = FALSE WHERE id = 1;
ROLLBACK TO SAVEPOINT before_change;
COMMIT;
```

Part 8 — Quick syntax reference

WANT TO...	PATTERN
See all rows	`SELECT * FROM table_name;`
See chosen columns	`SELECT col1, col2 FROM table_name;`
Filter rows	`SELECT * FROM table_name WHERE condition;`
Sort results	`SELECT * FROM table_name ORDER BY col1 DESC;`
Show a few rows	`SELECT * FROM table_name ORDER BY id LIMIT 10;`
Add a row	`INSERT INTO table_name (col1) VALUES ('value');`
Update a row	`UPDATE table_name SET col1 = 'value' WHERE id = 1;`
Delete a row	`DELETE FROM table_name WHERE id = 1;`
Count rows	`SELECT COUNT(*) FROM table_name;`
Combine related rows	`SELECT * FROM a JOIN b ON a.id = b.a_id;`
Group results	`SELECT col1, COUNT(*) FROM table_name GROUP BY col1;`
Hide soft-deleted	`SELECT * FROM table_name WHERE is_deleted = FALSE;`

Part 9 — Five beginner mistakes to avoid

1. **Forgetting `WHERE` in UPDATE or DELETE.** Without it, you may change or remove every row.
2. **Thinking FK values must be different from PK values.** A foreign key is supposed to match a referenced key value.
3. **Thinking `JOIN` permanently merges tables.** A JOIN combines related rows in a query result; the underlying tables remain separate.
4. **Thinking `is_deleted = TRUE` removes data.** It changes a flag; your queries must intentionally exclude those rows.
5. **Using `= NULL`.** Use `IS NULL` or `IS NOT NULL`.

> **Bonus:** Never concatenate untrusted user input directly into SQL strings. Use parameterized queries to prevent SQL injection.

Part 10 — Practice exercises

Q1. Show only student names.

Q2. Find the student whose ID is 102.

Q3. Show students in alphabetical order.

Q4. Find Rahul's partner's name using both tables.

Q5. Show all relationships that are not soft-deleted.

Q6. Count the number of students.

Answers

```sql
-- Q1
SELECT first_name FROM students;

-- Q2
SELECT * FROM students WHERE id = 102;

-- Q3
SELECT * FROM students ORDER BY first_name ASC;

-- Q4
SELECT b.first_name AS partner
FROM relationships AS r
JOIN students AS b ON b.id = r.student_b_id
WHERE r.student_a_id = 101 AND r.is_deleted = FALSE;

-- Q5
SELECT * FROM relationships WHERE is_deleted = FALSE;

-- Q6
SELECT COUNT(*) FROM students;
```

Part 11 — One-page memory map

Database
Organized collection of data.

Table
Rows and columns.

Primary Key
Unique ID for one row.

Foreign Key
Column referencing another table's key.

SELECT
Read data.

WHERE
Filter rows.

JOIN
Connect related records in query results.

INSERT
Add data.

UPDATE
Modify data.

DELETE
Remove rows.

Soft delete
Keep row, mark it inactive/hidden.

COMMIT / ROLLBACK
Save / undo a transaction.

If you understood how Rahul's ID 101 connects to a relationship row, how SQL retrieves Priya's name, and how UPDATE can hide the record without deleting it, you already understand the foundation of relational databases.

Keep learning, keep building. — Oye Nino

Educational note: All names, emails and records in this guide are fictional.

OYE NINO

Keep learning, keep building.

If this guide helped you, share it with a friend who's starting their SQL journey. More free guides, prompts and tools are waiting for you.

Follow Oye Nino

Instagram: **@oye.nino**

YouTube: **@oye_nino**

Website: **oyenino.com**

Free prompts, deploy guides, AI tools & more at **oyenino.com/prompts.html**